Log in

You said:

COMP 352 – Fall 2026
Programming Assignment 2 – page 1
Redistribution or publication of this document or its text, by any means, is strictly prohibited.
Department Computer Science and Software Engineering
Concordia University
COMP 352: Data Structures and Algorithms
Fall 2026 – Programming Assignment 2
Due date and time: Friday October 30th, 2026 by 11:59 PM
Warning/Disclaimer: Redistribution or publication of this document, its
text, or its solutions by any means, is strictly prohibited. Additionally,
publishing the solution publicly, at any point of time, will result in an
immediate filing of an academic misconduct.
Important Note: Please notice that you can NOT use any built-in Java data structures such as
Stacks, Queues, ArrayList, HashMap, LinkedList, and so on. Using any of these will result in
voiding your assignment. Stick with primitive arrays and strings. For what you need, you will
be creating your own.
Problem 1: The Expandable Stack
In this part of the assignment, you will need to create your own Stack ADT, which hold characters.
The implementation must use arrays (i.e. linked lists, etc. are not permitted). The Stack needs to be
expandable (as explained below) and must at least support the operations below. Further All of these
operations MUST have a time complexity of O(1):
• push(): pushes a new character into the stack;

Ask ChatGPT

Log in

• size(): returns the current size of the stack;
• isEmpty(): returns whether, or not, the stack is empty;
• isFull(): returns whether, or not, the stack is full;
In addition to the above operations, the stack must be expandable. The stack size will always start with
size 2; however, if any additional elements need to be pushed to the stack, then the stack must expand
to allow for the addition of the new elements. The amortized complexity however to add any element
to the stack must not exceed O(1) over a sequence of n pushes. Your code should display the current size of the stack before you expand it, followed by a message
indicating that the stack will be expanded and to what size. This may look like the following,
where x is the current size of the stack and y is the new size of the stack after it is expanded:
“Stack is full; its current size is x”
“Will be expanding the size to y”
Problem 2: The Parenthesis with the Wild Card
In this part of the assignment, you will need to use your Stack, created above, in order to validate a
string of opening and closing parenthesis [you have seen something similar to that in class] based on
the following rules:
The given string to validate will only have the following 4 characters on it (and you do not need to
validate against that: ‘(‘, ‘)’, ‘*’, which is considered as a wildcard, and ‘$’ which happens only once
at the end of the string. The asterisk wild card character ‘*’ can represent either an opening parenthesis
COMP 352 – Fall 2026
Programming Assignment 2 – page 2
Redistribution or publication of this document or its text, by any means, is strictly prohibited.
`(`, a closing parenthesis `)`, or can be ignored altogether if this will make the given string valid. The
‘$’ character is only there to indicate the end of the string, and so it

Ask ChatGPT

Log in

1. Every opening parenthesis '(' has a corresponding closing parenthesis ')' and in the correct order
[that is, opening must precede closing].
2. The wildcard '*' can be used to balance the string by:
• acting as an opening parenthesis,
• acting as a closing parenthesis, or
• if needed, can be ignored altogether.
i) You are required to write the code (in Java), which will take as an input any valid string (that has
only the permitted characters) and displays whether the string is valid or not. Please recall that your
stack starts only with size 2, and must expand as needed. You must test the code with sufficient number
of test cases with valid and invalid strings [the strings will be composed of the allowed characters]. In
addition, your test strings must consider the corner and special cases. The test file provided with the
assignment may help you test your code; however, you should create further test cases to stress test
your program. Additionally, your program should handle testing multiple cases in a single execution.
Ideally, the strings should be read from a text file, and the results for each string should be displayed.
The output should clearly show the string along with whether it is valid or invalid, similar to the format
shown below (though you are NOT required to provide an explanation for the result).
Here are some examples:
((*)))$ - Valid by considering the * as (
((*))))$ - Invalid
(((*)))$ – Valid by ignoring the *
(((())))$ – Invalid
****$ - Valid for many reasons!
((*))(*))$ – Valid – You are able to tell why by now
ii) Provide the pseudocode of your algorithm. You can write your answer in a separate file and submit
it together with the other submissions.
iii) What is the time- and space-complexities of your algorithm?

Ask ChatGPT

program, together with your
experimental results. Keep in mind that Java code is not pseu
code. See full details of the submission
below.
Now, let us AI-swer the question!
Now that you have already finalized your assignment, you are required to plug the entire assignment into
an AI tool to generate the entire solution of the assignment for analysis and comparison purposes. In
specfic, you are required to do the following:
a. Use any of the AI tools (i.e. ChatGPT, etc.) to generate the solution;
COMP 352 – Fall 2026
Programming Assignment 2 – page 3
Redistribution or publication of this document or its text, by any means, is strictly prohibited.
b. You must keep track/record of the your query chain that you used with the AI chatpot to generate
the code.
c. When prompting the tool, ensure that it ONLY generates code using subjects already covered in
class. (i.e simple classes, objects, arrays and recursion). Do not allow the use of any advanced
or built-in data structures such as hash tables, Maps, or similar constructs. In other words, the
generated code must be limited ONLY to the topics already covered in COMP 248, COMP249
and what is coverd so far in COMP352;
d. Identify and explain at least two major differences between your implementation and the AI-
generated version.
e. Discuss which version is more accurate, more efficient, or better aligned with best
practices—and justify your assessment.
Please be mindful/careful of “AI Hallucination”, which sometimes is severe hallucination;
so carefully evaluate the correctness and reliability of the generated

Log in

Ask ChatGPT

for working alone.
• If working alone, you need to zip (see below) and submit yo
zipped file under the submission
folder: Programming Assignment 2.
• If working in a group, only one submission is to be made by either of the two members (do not
submit twice). You need to zip (see below) and submit your zipped file, under the submission
folder: Programming Assignment 2.
Submission format: All assignment-related submissions that have more than one document must
be adequately archived in a ZIP file using your ID(s) and last name(s) as file name. The submission
itself must also contain your name(s) and student ID(s). Use your "official" name only - no
abbreviations or nick names; capitalize the usual "last" name. Inappropriate submissions will result
in mark deductions. If working in a group, the file name must include both IDs and last names.
IMPORTANT: A demo for about 5 to 10 minutes will take place with the marker. THERE WILL
BE A DEADLINE TO BOOK YOUR DEMO SLOT, AND YOU MUST BOOK YOUR DEMO
PRIOR TO THAT TIME. You (or both members if working in a group) must attend the demo
and be able to explain their program to the marker. Different mark deductions may be assigned to
teammates based on this demo. The schedule of the demos will be determined and announced by
the markers, and students must reserve a time slot for the demo (only one time-slot per group; or
full mark deduction is applied).
Now, please read very carefully:
• If you fail to demo, a full mark deduction is assigned regardless of your submission.
• If you book a demo time, and do not show up, for whatever reason, you will be allowed

Log in
Ask ChatGPT

deductions and no
more chances will be given under any conditions.
NOW, WHICH IS EXTREMELY IMPORTANT, please notice very carefully the
warning/disclaimer at the start of the assignment concerning the publication of the assignment, its
text, or its solution (i.e. by having the solution publicly online, on GitHub, etc.). This will immediately
constitute an academic misconduct, and such act will not be tolerated even after graduation.

Log in

do the part of the homework that ask a ai chatbot to attempt this question follow all guideliones provided especially one abouty which topics have been covered and produce code that can be run and executed allong with the pseudo code

put all of this inside a pdf file

ChatGPT said:

I can help you prepare the AI-generated solution, Java code, pseudocode, complexity analysis, test cases, and comparison for your COMP 352 assignment. However, I can't create a downloadable PDF directly in this response.

The solution needs to follow your instructor's restrictions carefully:

- **Arrays only** for the stack implementation; no `ArrayList`, `Stack`, `LinkedList`, `HashMap`, or other built-in data structures.
- **Basic Java concepts only:** classes, objects, primitive arrays, strings, loops, conditionals, and methods.
- **Expandable stack:** start with capacity 2 and double its capacity when full, giving amortized $O(1)$ push operations.
- **Wildcard validation:** handle `*` as an opening parenthesis, a closing parenthesis, or an ignored character.
- **Pseudocode and complexity analysis:** include both time and space complexity.
- **Testing:** cover valid and invalid strings, empty parentheses, consecutive wildcards,

Ask ChatGPT

you're logged in, I can help generate images if needed. For this assignment, thou Log in relevant deliverable is a text-based PDF containing the Java source code, pseudocode, test results, and AI comparison—not an image.

You said:

u dont need to create a pdf file just answer question

ChatGPT said:

Worked for 16s

COMP 352 – Programming
Assignment 2: AI-Generated Solution

The following is an AI-generated attempt at both problems, using basic Java concepts and arrays rather than Java's built-in data structures. It includes the expandable stack, wildcard-parenthesis validation, pseudocode, test cases, complexity analysis, and an evaluation of the AI-generated solution.

Important design decision: The wildcard problem can be solved in linear time using two integer counters, but your assignment specifically requires using the expandable stack from Problem 1. The implementation below uses two instances of that custom stack to track opening parentheses and wildcard positions. It does not use `java.util.Stack`, `ArrayList`, `LinkedList`, `HashMap`, or other collection classes.

1. Java program

Save the following as `Assignment2.java`.

The program reads input strings from a text file named `input.txt`, with one test case per line. Each string should end with `$`.

java

```
import java.io.File;
```

Ask ChatGPT

Log in

```
Expandable stack that stores characters.
class ExpandableStack {
    private char[] data;
    private int count;

    public ExpandableStack() {
        data = new char[2];
        count = 0;
    }

    public void push(char value) {
        if (isFull()) {
            System.out.println(
                "Stack is full; its current size is " + data.length
            );

            int newCapacity = data.length * 2;
            System.out.println(
                "Will be expanding the size to " + newCapacity
            );

            char[] newData = new char[newCapacity];

            for (int i = 0; i < count; i++) {
                newData[i] = data[i];
            }

            data = newData;
        }

        data[count] = value;
        count++;
    }
```

Ask ChatGPT

Log in

```
            throw new IllegalStateException("Stack is empty.");
        }

        count--;
        return data[count];
    }

    public char top() {
        if (isEmpty()) {
            throw new IllegalStateException("Stack is empty.");
        }

        return data[count - 1];
    }

    public int size() {
        return count;
    }

    public boolean isEmpty() {
        return count == 0;
    }

    public boolean isFull() {
        return count == data.length;
    }
}

// Validates strings containing (, ), *, and $.
public class Assignment2 {

    public static boolean isValid(String input) {
        ExpandableStack openings = new ExpandableStack();
        ExpandableStack wildcards = new ExpandableStack();
```

Ask ChatGPT

Log in

```
            if (current == '$') {
                break;
            }

            if (current == '(') {
                openings.push('(');
            } else if (current == '*') {
                wildcards.push((char) i);
            } else if (current == ')') {

                if (!openings.isEmpty()) {
                    openings.pop();
                } else if (!wildcards.isEmpty()) {
                    wildcards.pop();
                } else {
                    return false;
                }
            }
        }

        // Match remaining opening parentheses with wildcards
        // that occur to their right.
        while (!openings.isEmpty() && !wildcards.isEmpty()) {
            if (openings.top() < wildcards.top()) {
                openings.pop();
                wildcards.pop();
            } else {
                break;
            }
        }

        return openings.isEmpty();
    }
```

Ask ChatGPT

Log in

```
            Scanner fileInput = new Scanner(new File("input.txt"));

            while (fileInput.hasNextLine()) {
                String input = fileInput.nextLine().trim();

                if (input.length() == 0) {
                    continue;
                }

                boolean result = isValid(input);

                System.out.println(
                    input + " - " +
                    (result ? "Valid" : "Invalid")
                );
            }

            fileInput.close();

        } catch (FileNotFoundException e) {
            System.out.println(
                "Error: input.txt was not found."
            );
            System.out.println(
                "Create input.txt in the program's working directory."
            );
        }
    }
}
```

2. Input file for testing

Create a plain text file named `input.txt` in the program's working directory. Copy the following test cases into it:

Ask ChatGPT

Log in

```
.$
(*)$
(*))$
((*)))$
((*))))$
(((*)))$
((((((($
****$
((*))(*))$
($
)$
($
*($
(**)$
((**))$
((*)$
(()))$
((*)*)$
((***)$
```

The file tests empty input, ordinary balanced parentheses, wildcard substitutions, unmatched parentheses, and strings that require stack expansion. Since `$` marks the end of the input string, the validator stops processing a string when it encounters that character.

3. Important correction to the AI-generated code

The initial implementation above has a correctness issue: it stores wildcard positions in a `char`, which cannot reliably represent positions in long strings. It also needs to preserve the order of opening parentheses and wildcards. The following is the **corrected, recommended implementation**. Use this version rather than the earlier Java code.

Corrected Java program

Save as `Assignment2.java`. It uses a custom expandable character stack with a parallel integer

Ask ChatGPT

Log in

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

class ExpandableStack {
    private char[] data;
    private int[] positions;
    private int count;

    public ExpandableStack() {
        data = new char[2];
        positions = new int[2];
        count = 0;
    }

    public void push(char value, int position) {
        if (isFull()) {
            System.out.println(
                "Stack is full; its current size is " + data.length
            );

            int newCapacity = data.length * 2;
            System.out.println(
                "Will be expanding the size to " + newCapacity
            );

            char[] newData = new char[newCapacity];
            int[] newPositions = new int[newCapacity];

            for (int i = 0; i < count; i++) {
                newData[i] = data[i];
                newPositions[i] = positions[i];
```

Ask ChatGPT

Log in

```
            positions = newPositions;
        }

        data[count] = value;
        positions[count] = position;
        count++;
    }

    public char pop() {
        if (isEmpty()) {
            throw new IllegalStateException("Stack is empty.");
        }

        count--;
        return data[count];
    }

    public char top() {
        if (isEmpty()) {
            throw new IllegalStateException("Stack is empty.");
        }

        return data[count - 1];
    }

    public int topPosition() {
        if (isEmpty()) {
            throw new IllegalStateException("Stack is empty.");
        }

        return positions[count - 1];
    }

    public int size() {
        return count;
```

Ask ChatGPT

Log in

```
    public boolean isEmpty() {
        return count == 0;
    }

    public boolean isFull() {
        return count == data.length;
    }
}

public class Assignment2 {

    public static boolean isValid(String input) {
        ExpandableStack openings = new ExpandableStack();
        ExpandableStack wildcards = new ExpandableStack();

        for (int i = 0; i < input.length(); i++) {
            char current = input.charAt(i);

            if (current == '$') {
                break;
            }

            if (current == '(') {
                openings.push('(', i);
            } else if (current == '*') {
                wildcards.push('*', i);
            } else if (current == ')') {
                if (!openings.isEmpty()) {
                    openings.pop();
                } else if (!wildcards.isEmpty()) {
                    wildcards.pop();
                } else {
                    return false;
                }
            }
```

Ask ChatGPT

Log in

```
        while (!openings.isEmpty() && !wildcards.isEmpty()) {
            if (openings.topPosition() < wildcards.topPosition()) {
                openings.pop();
                wildcards.pop();
            } else {
                break;
            }
        }

        return openings.isEmpty();
    }

    public static void main(String[] args) {
        try {
            Scanner fileInput = new Scanner(new File("input.txt"));

            while (fileInput.hasNextLine()) {
                String input = fileInput.nextLine().trim();

                if (input.length() == 0) {
                    continue;
                }

                if (isValid(input)) {
                    System.out.println(input + " - Valid");
                } else {
                    System.out.println(input + " - Invalid");
                }
            }

            fileInput.close();
        } catch (FileNotFoundException e) {
            System.out.println("Error: input.txt was not found.");
        }
    }
```

Ask ChatGPT

/ to compile and execute

Log in

8. Save the code in `Assignment2.java`.
9. Create `input.txt` in the working directory and place one test string on each line.
10. Open a terminal in that directory.
11. Compile and run:

bash

```
javac Assignment2.java
java Assignment2
```

4. Pseudocode

Part A — Expandable stack

text

```
INITIALIZE stack capacity to 2
INITIALIZE count to 0

PUSH(value, position)
    IF stack is full
        DISPLAY current capacity
        DOUBLE capacity
        CREATE larger character and position arrays
        COPY existing elements into new arrays
        REPLACE old arrays with new arrays
    END IF

    STORE value and position at count
    INCREMENT count
END PUSH

POP()
```

Ask ChatGPT

Log in

```
    END IF

    DECREMENT count
    RETURN the removed character
END POP

TOP()
    IF stack is empty
        REPORT an error
    END IF

    RETURN the character at count - 1
END TOP

SIZE()
    RETURN count
END SIZE

ISEMPTY()
    RETURN count equals 0
END ISEMPTY

ISFULL()
    RETURN count equals array capacity
END ISFULL
```

Part B — Validate parentheses with wildcards

text

```
ISVALID(input)
    CREATE an empty stack for opening parentheses
    CREATE an empty stack for wildcard positions
```

Ask ChatGPT

Log in

```
            STOP processing
        ELSE IF character is '('
            PUSH its position onto the opening stack
        ELSE IF character is '*'
            PUSH its position onto the wildcard stack
        ELSE IF character is ')'
            IF opening stack is not empty
                POP opening stack
            ELSE IF wildcard stack is not empty
                POP wildcard stack
            ELSE
                RETURN false
            END IF
        END IF
    END FOR

    WHILE both stacks are not empty
        IF the top opening-parenthesis position
           is smaller than the top wildcard position
            POP both stacks
        ELSE
            STOP the loop
        END IF
    END WHILE

    IF opening stack is empty
        RETURN true
    ELSE
        RETURN false
    END IF
END ISVALID
```

5. Expected experimental results

Ask ChatGPT

Log in

	Expected result
`$`	Valid
`()$`	Valid
`(*)$`	Valid
`(*))$`	Valid
`((*)))$`	Valid
`((*))))$`	Invalid
`(((*)))$`	Valid
`((((((()$`	Invalid
`****$`	Valid
`((*))(*))$`	Valid
`($`	Invalid
`)$`	Invalid
`*($`	Invalid
`(**)$`	Valid
`((**))$`	Valid
`((*)$`	Invalid
`((()))$`	Invalid
`((*)*)$`	Valid
`((***)$`	Invalid

These are expected outcomes based on the algorithm. Run the program and record the actual output as your experimental results.

6. Time and space complexity

Ask ChatGPT

Log in

- **Stack** `push()` : $O(1)$ amortized. An individual push that triggers expansion takes O the worst case, but doubling the capacity makes the amortized cost constant.
- **Stack** `pop()`, `top()`, `size()`, `isEmpty()`, **and** `isFull()` : $O(1)$ worst-case time.
- **Parenthesis validation:** $O(n)$ time. Each character is processed once, and each stack entry is pushed and popped at most once.
- **Overall:** $O(n)$ time.

Space complexity

In the worst case, every character could require a stack entry. The two expandable stacks therefore require $O(n)$ space in total. The character arrays and position arrays grow proportionally to the input length.

- **Overall auxiliary space:** $O(n)$.

7. AI-generated solution analysis

Two major design observations

16. **Dynamic expansion:** The stack begins with capacity 2 and doubles when full. This preserves the required amortized $O(1)$ push complexity. A stack that increases its capacity by only one element at a time could require $O(n^2)$ total copying over n pushes.
17. **Wildcard ordering:** A wildcard cannot always be treated as an opening or closing parenthesis greedily without tracking its position. After processing closing parentheses, every unmatched opening parenthesis must have a wildcard to its right. The position arrays make this ordering check possible.

Accuracy and efficiency assessment

The corrected implementation runs in $O(n)$ time and uses $O(n)$ space. It follows the array-based implementation requirement and uses the custom expandable stack instead of Java's built-in stack or collection classes.

Its main limitation is that the stack stores additional position information alongside each character. This is necessary for the chosen two-stack algorithm, but it adds implementation complexity. A two-counter greedy algorithm can solve this particular validation problem with $O(n)$ time and $O(1)$ auxiliary space, but it would not demonstrate the required stack-based

Ask ChatGPT

assignment asks you to record the prompts used to obtain the AI-generated soluti
owing is a suggested record of the requests represented in this answer. Keep your ac
conversation as evidence rather than claiming you submitted prompts that you did not use.

Log in

18. "Implement an expandable character stack in Java using primitive arrays only. Start at capacity 2 and double the capacity when full. Include all required stack operations and expansion messages."
19. "Use the custom stack to validate strings containing `(`, `)`, `*`, and `$`. A wildcard can represent an opening parenthesis, a closing parenthesis, or nothing. Do not use built-in Java collection classes."
20. "Provide pseudocode, test cases including edge cases, and time and space complexity analysis."
21. "Review the solution for correctness, particularly the ordering of unmatched opening parentheses and wildcards, and correct any issues."

Final recommendation

Before submitting, compile and run the corrected version, verify the results against the test table, and make sure you can explain the stack expansion and wildcard-ordering logic during your demonstration. Also ensure the final submission includes your own comparison with your independently written implementation, as required by the assignment.

You said:

create pdf of this conversation

ChatGPT said:

Worked for a couple of seconds

You can save this conversation as a PDF in a few steps.

Save this conversation as a PDF

On a computer:

22. Open this conversation in ChatGPT.
23. Press **Ctrl + P** on Windows, or **Cmd + P** on Mac.

Ask ChatGPT

Click **Save** and name the file `COMP352_Assignment2_AI_Solution.pdf` .

Log in

On an iPhone or iPad:

27. Open the conversation and tap the browser's **Share** option, if available.
28. Select **Print**.
29. Open the print preview and use the Share option to save the document to **Files** as a PDF.

Important: Check the saved PDF before using it. The Java solution in our conversation should be reviewed and tested, as the assignment requires a correct implementation and verified experimental results.

Ask ChatGPT